

Assignment 2 Report

LSTM Autoencoder Anomaly Detection

Student: Mohamad Rami Karkour

mokr22@student.bth.se

Course: DV2586 Deep Learning
Dataset: NSL-KDD Network Intrusion Detection

1. Overview

For this assignment, I worked with anomaly detection using the NSL-KDD network intrusion dataset. I treated normal network traffic as the normal class and all attack traffic as anomalies. The main idea was to train LSTM autoencoders only on normal traffic, let them learn what normal sequences look like, and then flag sequences as anomalous when the reconstruction error became too high.

I also compared the LSTM models with an Isolation Forest baseline. That comparison was important because NSL-KDD is mainly a tabular network dataset, not a natural sensor time series. So I wanted to see whether the LSTM sequence approach actually helped, or whether a traditional anomaly detection method would perform almost the same.

2. Dataset and preprocessing

The original training file contained 125,973 rows and the original test file contained 22,544 rows. There were no missing values in the processed feature tables. The labels were converted into a binary format: normal traffic became 0 and every attack type became 1.

Categorical columns were one-hot encoded, and the final input dimension became 122 features. The scaler was fitted on normal training rows only. I did this because, in an anomaly detection setting, the model should learn the scale of normal behavior without using attack rows during preprocessing.

The sequence length was set to 10. The normal training set produced 60,600 sequences and the validation set produced 6,734 sequences. The mixed test data was split into 6,760 tuning sequences and 15,775 final test sequences. The final test anomaly ratio was about 56.9%.

Because NSL-KDD is tabular, I created sequences using a sliding window. For the mixed test sequences, the label of the last row in the window was used as the sequence label. This avoids turning nearly every window into an anomaly just because one earlier row in the window was an attack.

3. LSTM autoencoder models

I trained two LSTM autoencoders. The first one was smaller, with one LSTM layer, hidden size 64, and latent dimension 32. The second one was deeper, with two LSTM layers, hidden size 128, dropout, and the same latent dimension. So the second model had more capacity and more compression pressure.

Both models follow the same general autoencoder idea. The encoder compresses the input sequence into a latent vector, and the decoder tries to reconstruct the original sequence. If the input sequence is normal, the reconstruction error should usually be lower. If it is unusual or attack-like, the error should often be higher.

The reconstruction error was calculated as the mean squared error over all time steps and all features in the sequence:

$$E_i = \frac{1}{(T \times F)} \sum_t \sum_j (x_{i,t,j} - x_{\text{hat}_{i,t,j}})^2$$

Here, T is the sequence length and F is the number of features. I used the average error because it gives one simple anomaly score for each sequence. A stricter alternative would be to use the maximum timestep error, which might be useful in a live network monitoring setup.

4. Training setup

The autoencoders were trained only on normal sequences. I used AdamW, 40 epochs, and early stopping logic based on validation loss. The losses kept improving overall, so the full training schedule was useful in this run.

The training curves show that both models learned to reconstruct normal sequences. The curves are not perfectly smooth, especially for the deeper model, but the overall direction is clearly downward. The deeper model reached lower validation loss, although that did not automatically make it the best anomaly detector.

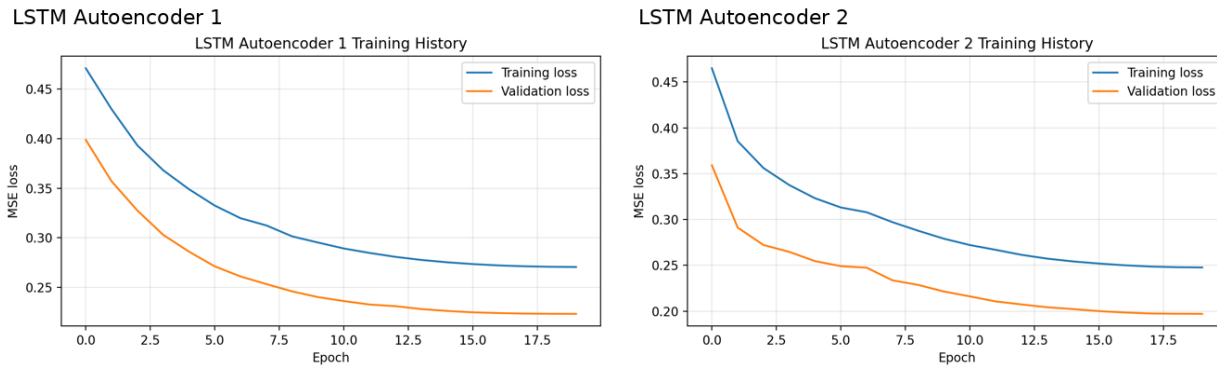


Figure 1. Training and validation loss for the two LSTM autoencoders.

5. Threshold selection and evaluation

The anomaly threshold was selected on the tuning split, not directly on the final test split. I swept possible threshold values and selected the one that gave the best F1-score on the tuning data. Then I used that fixed threshold on the final test split. This made the evaluation cleaner than choosing the threshold after seeing final test performance.

The Isolation Forest baseline used the same final evaluation logic. The sequence data was flattened, the model was trained on normal training sequences, and its anomaly score threshold was chosen using the tuning split.

6. Results

Model	Prec.	Recall	F1	ROC-AUC	AUC-PR	TP	TN	FP	FN
LSTM AE 1	0.584	0.968	0.728	0.596	0.625	8692	602	6193	288
LSTM AE 2	0.582	0.969	0.727	0.595	0.625	8706	541	6254	274
Isolation Forest	0.581	0.965	0.725	0.595	0.644	8670	535	6260	310

The small LSTM autoencoder gave the best F1-score, 0.728. The deeper LSTM model was very close at 0.727. Isolation Forest had a slightly lower F1-score, but it had the strongest AUC-PR score at 0.644.

The clearest pattern is high recall. All three models detected around 96-97% of anomalous sequences. The weaker side is precision, which stayed around 58%. In other words, the models were sensitive and caught most attacks, but they also flagged many normal sequences as anomalies.

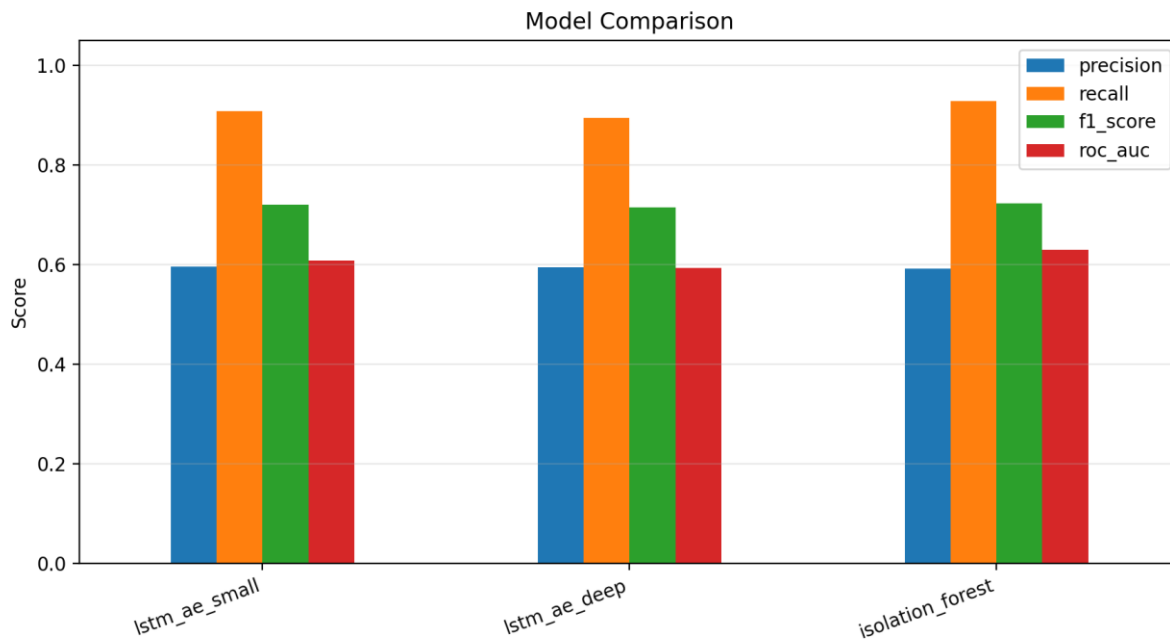
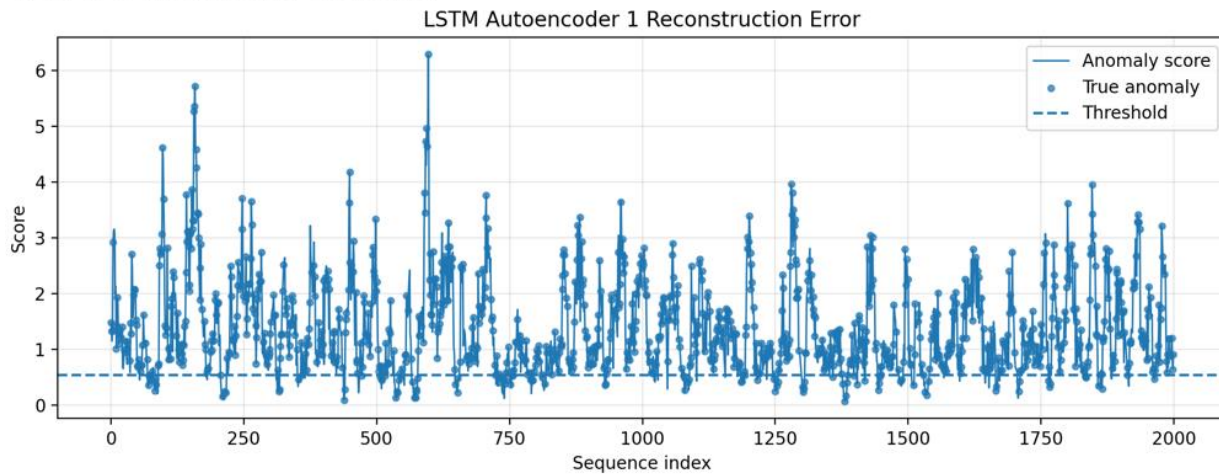


Figure 2. Model comparison across precision, recall, F1-score, ROC-AUC, and AUC-PR.

7. Visual analysis

The reconstruction-error plots show how the final anomaly decision was made. Scores above the threshold are treated as anomalies. Many attack sequences have high scores, which explains the strong recall, but some normal sequences also cross the threshold, which explains the false positives.

LSTM AE 1 reconstruction error



LSTM AE 2 reconstruction error

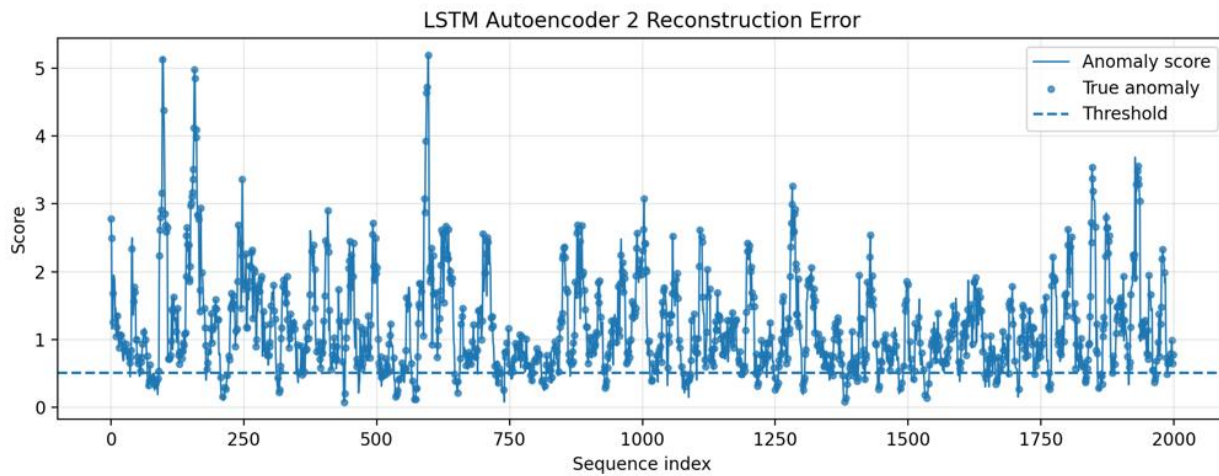


Figure 3. Reconstruction error over the first 2,000 final-test sequences.

The histograms make the limitation clearer. Normal and anomaly score distributions overlap quite a lot. The anomaly scores are shifted slightly, but the overlap is still large. This is why the ROC-AUC values stay around 0.60 instead of becoming much higher.

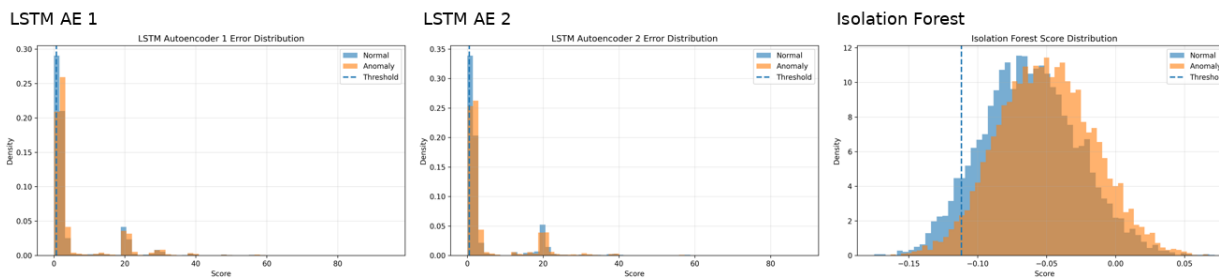


Figure 4. Score distributions for normal and anomalous sequences.

The confusion matrices show the same thing from another angle. The models produced many true positives, but also many false positives. This means the anomaly signal is useful, but the separation is not clean enough to avoid many false alarms.

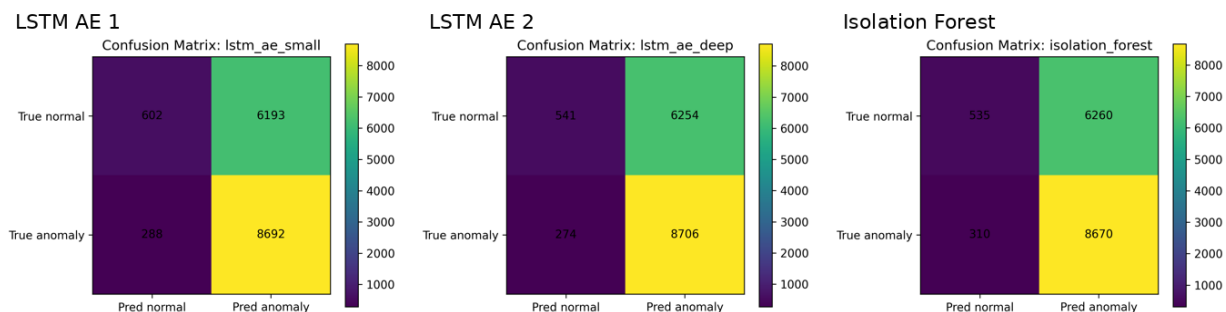


Figure 5. Confusion matrices for both LSTM models and the Isolation Forest baseline.

8. Discussion

One thing I found interesting is that the deeper LSTM had lower validation loss, but it did not beat the smaller model in final F1-score. This shows that better reconstruction of normal validation data does not always mean better anomaly detection. What really matters is how well the error scores separate normal and anomalous sequences.

Isolation Forest was also competitive. I think this makes sense because NSL-KDD is mainly tabular. The LSTM models use sliding windows to create sequence structure, but this is still not the same as working with a real time-series dataset. That probably limited how much the LSTM models could outperform the baseline.

The current models are strongest when the goal is to catch as many anomalies as possible. The recall is high, so relatively few attacks are missed. However, the false-positive rate is also high. In a real network system, that would create many alerts, so the threshold policy would need more work.

9. Conclusion

Overall, the experiment showed that LSTM autoencoders can be used for anomaly detection on network intrusion data when trained only on normal traffic. The small LSTM autoencoder gave the best F1-score, while Isolation Forest had the strongest AUC-PR score. The models did not separate normal and anomalous traffic perfectly, but they detected most attacks and produced meaningful anomaly scores.

My final interpretation is that the LSTM approach is valid, but the dataset structure limits the result. The sliding-window setup gives some local context, but NSL-KDD is still not a true time-series dataset. Because of that, I would treat this as a solid anomaly-detection baseline rather than a finished intrusion detection system.